



Práctica 1B

Métrica intra/inter clusters

Aprendizaje No Supervisado y Visualización
Unidad 1 · Maestría en Ciencia de Datos e Información

Bryan Rodríguez Abarca

Matrícula 26DF1185A9

20 de agosto de 2026

La práctica pide implementar la métrica que compara la distancia entre datos del mismo cluster contra la distancia entre datos de clusters distintos, y usarla para calificar cuatro particiones distintas del mismo conjunto de datos. La función recibe los datos, sus etiquetas y cuántos pares muestrear, y devuelve un solo número.

Todas las cifras que aparecen en el texto salen de ejecutar este notebook.

1. Qué mide la métrica

Un agrupamiento está bien hecho si los datos que quedaron juntos se parecen entre sí más de lo que se parecen a los que quedaron en otro grupo. La métrica intra/inter convierte esa idea en un número: se toman pares de datos al azar, cada par se clasifica según si sus dos elementos comparten cluster (*intra*) o no (*inter*), y se promedian por separado las distancias intra y las inter. La medida es el cociente entre ambos promedios.

$$\text{intra} = \frac{1}{|P|} \sum_{(i,j) \in P} d(x_i, x_j) \quad \text{inter} = \frac{1}{|Q|} \sum_{(i,j) \in Q} d(x_i, x_j) \quad \text{medida} = \frac{\text{intra}}{\text{inter}}$$

con P los pares que comparten cluster, Q los que no y d la distancia euclidiana. Valores pequeños indican mejor calidad; un valor cercano a 1 dice que dos datos del mismo grupo están, en promedio, tan lejos como dos datos de grupos distintos.

Se muestrea porque el número de pares crece con el cuadrado de los datos y a partir de cierto tamaño calcularlos todos se encarece. Con estos 1797 no es el caso: el cálculo exhaustivo me tomó poco más de un segundo, casi todo en armar el millón y medio de distancias, y las cuatro métricas sobre ellas salen en cincuenta milésimas, según el cronómetro que pongo más abajo. Los 500 pares son entonces la convención del enunciado y no una necesidad. El muestreo toma índices con reemplazo y sin ordenar el par, de modo que el espacio del que salen esos 500 son los 1797^2 pares ordenados, no los pares distintos que también imprimo.

```
import numpy as np
import sklearn, sys

print("python      ", sys.version.split()[0])
print("numpy       ", np.__version__)
print("scikit-learn", sklearn.__version__)
```

```
python      3.11.15
numpy       2.4.6
scikit-learn 1.9.0
```

2. Los datos y las cuatro particiones

Antes de implementar nada revisé qué son X y las cuatro y , porque el resultado de la métrica no se puede leer sin saber cuántos grupos hay y de qué tamaño. El bloque del enunciado los vuelve a cargar más abajo.

Los 64 números de cada dato me hicieron pensar en el conjunto de dígitos escritos a mano de scikit-learn, que son imágenes de 8 por 8, pero los valores no coincidían: los de `load_digits` llegan a 16 y los de `X` a 1. Probé dividir entre 16 y la comparación cuadró exacta.

```
from sklearn.datasets import load_digits
import textwrap

X = np.load("X.npy")
ys = [np.load(f"y_{i}.npy") for i in range(4)]

d = load_digits()
print("X:", X.shape, X.dtype, "rango", X.min(), "a", X.max())
print("¿X es load_digits().data / 16?",
      np.allclose(X, d.data / 16.0))
print("pares ordenados:", len(X) ** 2,
      " pares distintos:", len(X) * (len(X) - 1) // 2)
print()

for i, y in enumerate(ys):
    u, c = np.unique(y, return_counts=True)
    print(f"y_{i}: {len(u):2d} etiquetas distintas, de {u.min()} a "
          f"{u.max()} ruido (-1): {(y == -1).sum()}")
    print(textwrap.fill(str(np.sort(c)[::-1].tolist()), 62,
                        initial_indent="    tamaños: ",
                        subsequent_indent=" " * 15))
```

```
X: (1797, 64) float64 rango 0.0 a 1.0
¿X es load_digits().data / 16? True
pares ordenados: 3229209 pares distintos: 1613706

y_0: 10 etiquetas distintas, de 0 a 9 ruido (-1): 0
     tamaños: [296, 273, 222, 188, 180, 179, 170, 142, 108,
               39]
y_1: 13 etiquetas distintas, de -1 a 11 ruido (-1): 67
     tamaños: [1166, 178, 164, 160, 67, 27, 12, 8, 4, 4, 3,
               2, 2]
y_2: 22 etiquetas distintas, de -1 a 20 ruido (-1): 497
     tamaños: [497, 176, 163, 159, 148, 148, 113, 82, 68, 49,
               44, 35, 27, 22, 14, 11, 8, 7, 7, 7, 7, 5]
y_3: 5 etiquetas distintas, de 0 a 4 ruido (-1): 0
     tamaños: [565, 378, 359, 317, 178]
```

Las cuatro particiones son muy distintas entre sí. `y_0` reparte los 1797 datos en 10 grupos: nueve entre 108 y 296 elementos, más uno de 39, y sin ninguna etiqueta negativa. `y_3` usa 5 grupos, también sin negativos y con tamaños entre 178 y 565.

Las otras dos traen etiquetas -1; asumo que son puntos sin asignar, la convención de DBSCAN. `y_1` tiene 67 de esos, pero su rasgo dominante es otro: un solo grupo se lleva 1166 de los 1797 datos,

el 65 %, y ocho de sus grupos tienen menos de 30 elementos. `y_2` va al otro extremo, con 497 datos sin asignar —más de la cuarta parte— repartiendo el resto en 21 grupos.

3. La implementación

3.1. Qué hago con las etiquetas -1

Aquí tuve que decidir algo que el enunciado no dice. La función recibe `X` y `y` y nada más; nada en el código dado señala que alguna etiqueta sea especial, así que la única lectura que se sostiene con la firma que me piden es la literal: dos datos están en el mismo cluster cuando su etiqueta coincide, y eso incluye el par formado por dos datos etiquetados `-1`. Es lo que implemento y es el número que reporto.

Dejo dicho desde ya que esa decisión no es inocua, porque tratar el ruido como si fuera un grupo mete en el montón *intra* pares de datos que justamente no se parecen a nada. Más abajo mido cuánto cambia cada score con la lectura contraria, la de descartar los pares que tocan un `-1`.

3.2. El código

Traduzco la definición casi palabra por palabra. `pairs` es un arreglo de 500 renglones por 2 columnas, así que `pairs[:, 0]` son los índices del primer elemento de cada par y `pairs[:, 1]` los del segundo; `X[pairs[:, 0]]` recoge esos 500 datos en el orden en que aparecen. Restar los dos arreglos da las 500 diferencias de golpe, y la raíz de la suma de sus cuadrados por renglón da las 500 distancias. `mismo` marca los pares que comparten etiqueta y con él parto las distancias en los dos montones.

Si una partición dejara vacío uno de los montones —todos los pares intra o todos inter— el promedio de un arreglo vacío devolvería `nan` con una advertencia; con estas cuatro cada montón tiene pares.

```
import numpy as np

def intra_inter(X, y, num_pairs):
    np.random.seed(0)
    pairs = np.random.randint(len(X), size = (num_pairs, 2))
    a = X[pairs[:, 0]]
    b = X[pairs[:, 1]]
    d = np.sqrt(((a - b) ** 2).sum(axis=1))
    mismo = y[pairs[:, 0]] == y[pairs[:, 1]]
    return d[mismo].mean() / d[~mismo].mean()

X = np.load("X.npy")
ys = []
for i in range(4):
    ys.append(np.load(f"y_{i}.npy"))

for p in ys:
    print(intra_inter(X, p, 500))
```

```
0.740514037559034
0.9081780547227971
0.8202116394020493
0.8414202500259702
```

4. Los cuatro scores

Ordenados de menor a mayor, `y_0` sale en 0.7405, `y_2` en 0.8202, `y_3` en 0.8414 y `y_1` en 0.9082. Como valores chicos son mejores, la lectura inmediata es que `y_0` es la partición más compacta de las cuatro y `y_1` la que menos dice sobre las distancias.

Ese resultado cuadra con lo que vi al inspeccionar las etiquetas. `y_0` tiene 10 grupos pares y ninguna etiqueta de ruido, que es lo que uno esperaría de una partición sana de un conjunto con diez dígitos. `y_1` concentra el 65 % de los datos en un solo grupo, y como ese grupo por sí solo es más de medio conjunto, un par intra de `y_1` se parece bastante a un par tomado al azar; por eso es el que más se acerca a 1.

`y_2` y `y_3` quedan en medio y muy cerca uno del otro, con 0.02 de diferencia. Esa diferencia se voltea al cambiar la muestra de pares, como se ve más abajo, así que no los leo como ordenados entre sí.

5. Cuánto pesa la decisión sobre el ruido

Con los cuatro scores el enunciado queda cerrado; lo que sigue son extras propios. Ninguno modifica la función entregada: cuando necesito el desglose reproduzco su muestreo por fuera, con la misma semilla y el mismo tamaño.

El score de `y_2` me dio 0.8202 y no me cuadraba con sus tamaños de grupo: si `y_2` parte los datos en 21 grupos chicos, sus pares intra deberían ser muy cercanos y el cociente bastante más bajo. Fui a ver de dónde salían esos pares intra y ahí apareció el efecto del -1. De paso separo los pares que caen en el grupo mayor de cada partición, para ver si el score de `y_1` sale de donde creo que sale.

```
np.random.seed(0)
pairs = np.random.randint(len(X), size=(500, 2))
dist = np.sqrt(((X[pairs[:, 0]] - X[pairs[:, 1]]) ** 2).sum(axis=1))
print(f"distancia media de los 500 pares: {dist.mean():.4f}")
print()

for i, y in enumerate(ys):
    mismo = y[pairs[:, 0]] == y[pairs[:, 1]]
    ruido = mismo & (y[pairs[:, 0]] == -1)
    libre = (y[pairs[:, 0]] != -1) & (y[pairs[:, 1]] != -1)
    sin = dist[libre]
    msin = mismo[libre]
    alt = sin[msin].mean() / sin[~msin].mean()
    u, c = np.unique(y, return_counts=True)
    may = u[c.argmax()]
    eng = mismo & (y[pairs[:, 0]] == may)
    print(f"y_{i}: intra {mismo.sum():3d} pares, de ellos {ruido.sum():3d} "
          f"son -1 con -1")
```

```

print(f"        literal {dist[mismo].mean() / dist[~mismo].mean():.4f}"
      f"        descartando pares con -1 {alt:.4f}")
print(f"        grupo mayor: etiqueta {may} con {c.max()} datos, "
      f"{eng.sum()} pares intra "
      f"({100 * eng.sum() / mismo.sum():.1f}%)")
print(f"        distancia media dentro de ese grupo: "
      f"{dist[eng].mean():.4f}")
if ruido.sum() > 0:
    print(f"        distancia media -1 con -1: "
          f"{dist[ruido].mean():.4f}    resto de los intra: "
          f"{dist[mismo & ~ruido].mean():.4f}")

```

distancia media de los 500 pares: 3.0226

```

y_0: intra 53 pares, de ellos 0 son -1 con -1
      literal 0.7405   descartando pares con -1 0.7405
      grupo mayor: etiqueta 1 con 296 datos, 8 pares intra (15.1 %)
      distancia media dentro de ese grupo: 2.0696
y_1: intra 212 pares, de ellos 1 son -1 con -1
      literal 0.9082   descartando pares con -1 0.9026
      grupo mayor: etiqueta 1 con 1166 datos, 196 pares intra (92.5 %)
      distancia media dentro de ese grupo: 2.9049
      distancia media -1 con -1: 3.6422    resto de los intra: 2.8526
y_2: intra 60 pares, de ellos 35 son -1 con -1
      literal 0.8202   descartando pares con -1 0.6448
      grupo mayor: etiqueta -1 con 497 datos, 35 pares intra (58.3 %)
      distancia media dentro de ese grupo: 2.8996
      distancia media -1 con -1: 2.8996    resto de los intra: 2.0219
y_3: intra 109 pares, de ellos 0 son -1 con -1
      literal 0.8414   descartando pares con -1 0.8414
      grupo mayor: etiqueta 2 con 565 datos, 46 pares intra (42.2 %)
      distancia media dentro de ese grupo: 2.8308

```

El caso de `y_2` es el que se mueve: 35 de sus 60 pares intra son pares de dos datos marcados como ruido, y su distancia media es 2.8996 contra 2.0219 de los pares intra que sí comparten un grupo real. Como la distancia media de los 500 pares es 3.0226, esos 35 pares están aportando al promedio *intra* algo casi indistinguible de un par al azar. Descartándolos el score de `y_2` cae a 0.6448, por debajo del de `y_0`.

En `y_1` el cambio es de milésimas, 0.9082 contra 0.9026, porque solo 67 de sus 1797 datos son ruido. Lo que sí se confirma es de dónde viene su score: 196 de sus 212 pares intra, el 92.5 %, caen dentro del grupo de 1166 datos, y la distancia media ahí es 2.9049, muy cerca del 3.0226 de un par cualquiera. En `y_0` y `y_3` no hay etiquetas negativas y el número no se mueve.

Me quedo con la lectura literal para la entrega, porque es la que se desprende del código dado, y el score de `y_2` queda registrado junto con el dato de que una cuarta parte de sus datos no está agrupada.

6. Firmeza del valor con 500 pares

500 pares es una muestra muy chica frente a lo que hay, así que quise ver si el número ya se estabilizó ahí. Vuelvo a llamar a la función entregada cambiando solo `num_pairs`. Como la semilla es siempre 0, las muestras están anidadas: los primeros 100 pares de la corrida de 200 son los mismos 100 de la corrida anterior, así que los renglones de la tabla no son estimaciones independientes sino una sola muestra que va creciendo.

```
tam = [100, 200, 500, 1000, 5000, 20000, 100000]

print("num_pairs      y_0      y_1      y_2      y_3")
for n in tam:
    fila = [intra_inter(X, y, n) for y in ys]
    print(f"{n:9d}    " + " ".join(f"{v:.4f}" for v in fila))
```

num_pairs	y_0	y_1	y_2	y_3
100	0.8107	0.9301	0.8431	0.8228
200	0.7956	0.9213	0.8832	0.8030
500	0.7405	0.9082	0.8202	0.8414
1000	0.7355	0.9177	0.8256	0.8357
5000	0.7217	0.9277	0.8139	0.8331
20000	0.7225	0.9292	0.8245	0.8291
100000	0.7272	0.9322	0.8236	0.8292

Con 100 y 200 pares los valores todavía brincan en centésimas: `y_0` sale 0.8107 y termina cerca de 0.727, y `y_2` pasa de 0.8431 a 0.8832 entre 100 y 200 pares. Es lo esperable cuando el promedio intra se calcula con un puñado de distancias, que es lo que quedaba en `y_0` con 500 pares y todavía menos con 100. De 20000 en adelante los cambios ya son de milésimas.

`y_0` es el más bajo y `y_1` el más alto en los siete tamaños, así que esa parte los 500 pares del enunciado la resuelven bien. Lo que no resuelven es la pareja de en medio: con 100 y 200 pares `y_3` queda por debajo de `y_2`, y de 500 en adelante se invierten, con una separación que llega a 0.0212 en el propio renglón de 500 y baja a 0.006 con 100000 pares.

7. El cálculo exhaustivo y una partición de control

Los pares distintos que se pueden formar con 1797 datos caben en memoria, así que calculé la métrica con todos ellos para ver a qué valor converge el muestreo.

Además me faltaba una referencia. Leyendo que `y_3` da 0.8414 no sabía si eso es bueno o malo, porque no tenía contra qué compararlo. Lo que se me ocurrió fue barajar las etiquetas de cada partición: eso conserva exactamente el número de grupos y sus tamaños, y rompe cualquier relación con los datos, así que da el valor que sacaría una partición con ese mismo reparto pero sin información dentro.

```
import time

t0 = time.perf_counter()
i, j = np.triu_indices(len(X), k=1)
```

```

D = np.sqrt(((X[i] - X[j]) ** 2).sum(axis=1))
t_D = time.perf_counter() - t0

def exhaustivo(y):
    m = y[i] == y[j]
    return D[m].mean() / D[~m].mean()

t0 = time.perf_counter()
ex = [exhaustivo(y) for y in ys]
print(f"{len(D)} distancias en {t_D:.2f} s, y los 4 scores "
      f"exhaustivos en {time.perf_counter() - t0:.3f} s")
print()
print("          y_0      y_1      y_2      y_3")
print("exhaustivo: " + " ".join(f"{v:.4f}" for v in ex))
print("con 500    : " + " ".join(f"{intra_inter(X, y, 500):.4f}"
                                for y in ys))
print()

rng = np.random.default_rng(0)
print("etiquetas barajadas, 3 repeticiones por partición:")
for k, y in enumerate(ys):
    nulo = [exhaustivo(rng.permutation(y)) for _ in range(3)]
    print(f"y_{k}: " + " ".join(f"{v:.4f}" for v in nulo))

```

1613706 distancias en 1.11 s, y los 4 scores exhaustivos en 0.051 s

	y_0	y_1	y_2	y_3
exhaustivo:	0.7309	0.9318	0.8260	0.8316
con 500 :	0.7405	0.9082	0.8202	0.8414

etiquetas barajadas, 3 repeticiones por partición:

y_0:	1.0004	0.9997	0.9994
y_1:	0.9973	1.0047	1.0025
y_2:	0.9986	0.9981	0.9986
y_3:	1.0007	0.9990	1.0003

Los doce valores de control caen entre 0.9973 y 1.0047, o sea que cada una de estas cuatro particiones da 1 cuando le quito la información y le dejo el reparto de tamaños. Con eso los cuatro scores se leen como distancia de su propio control, y las cuatro particiones están por debajo del suyo, y_1 incluida: 0.9318 contra 1.000, poca separación pero separación.

El exhaustivo confirma también la pareja de en medio: y_2 en 0.8260 y y_3 en 0.8316 difieren en 0.0056, bastante menos de lo que se mueven al cambiar la semilla del muestreo.

8. Casos degenerados del muestreo

Como `np.random.randint` muestrea con reemplazo, en los 500 pares pueden salir un par de un dato consigo mismo, que aporta distancia 0 al montón intra, y pares repetidos, que cuentan doble. Antes de confiar en el número fui a contarlos. Ordeno los dos índices dentro de cada renglón antes de contar, porque el par (7, 300) y el (300, 7) son el mismo par de datos y sin ordenarlos se cuentan como distintos.

Y como la semilla del muestreo está fijada en 0 dentro de la función, todo el resultado depende de esos 500 pares en particular. Para ver si el orden de las cuatro particiones es una propiedad de los datos o de la muestra, repito el cálculo con otras semillas.

```
np.random.seed(0)
p0 = np.random.randint(len(X), size=(500, 2))
iguales = (p0[:, 0] == p0[:, 1]).sum()
unicos = len(np.unique(np.sort(p0, axis=1), axis=0))
print("pares de un dato consigo mismo:", iguales)
print("pares repetidos:", 500 - unicos)
print()

print("semilla      y_0      y_1      y_2      y_3")
for s in range(6):
    np.random.seed(s)
    pr = np.random.randint(len(X), size=(500, 2))
    dr = np.sqrt(((X[pr[:, 0]] - X[pr[:, 1]]) ** 2).sum(axis=1))
    fila = []
    for y in ys:
        m = y[pr[:, 0]] == y[pr[:, 1]]
        fila.append(dr[m].mean() / dr[~m].mean())
    print(f"{s:7d}  " + " ".join(f"{v:.4f}" for v in fila))
```

```
pares de un dato consigo mismo: 0
pares repetidos: 0
```

```
semilla      y_0      y_1      y_2      y_3
0  0.7405  0.9082  0.8202  0.8414
1  0.7381  0.9091  0.8193  0.8321
2  0.7557  0.9433  0.8701  0.8501
3  0.7501  0.9388  0.8634  0.8337
4  0.7664  0.9384  0.7979  0.8459
5  0.7058  0.9349  0.8219  0.8407
```

En los 500 pares de la semilla 0 no cayó ningún par de un dato consigo mismo ni ningún par repetido, así que no hizo falta filtrar nada y el número entregado no está contaminado por eso. Con 1797 datos el par consigo mismo tiene probabilidad $1/1797$, de modo que en 500 tiros esperaríamos menos de uno; salir cero es lo normal, no una garantía del método.

El barrido de semillas deja `y_0` como el más bajo y `y_1` como el más alto en las seis. `y_2` y `y_3` se cruzan: con la semilla 2 `y_2` queda 0.02 arriba de `y_3` y con la semilla 4 queda 0.05 abajo.

9. Conclusión

De las cuatro particiones, y_0 es la mejor calificada por la métrica: 0.7405 con los 500 pares del enunciado y 0.7309 con todos los pares, el valor más bajo en los siete tamaños de muestra y en las seis semillas que probé. y_1 es la peor, 0.9082 con 500 pares y 0.9318 exhaustivo, y su forma explica buena parte del resultado: con 1166 de sus 1797 datos en un solo grupo, el 92.5 % de los pares intra sale de ahí y la distancia media dentro de ese grupo (2.9049) queda cerca de la de un par cualquiera (3.0226), así que a la métrica le queda poco margen para bajar. Aun así se aleja de su control barajado, que da 1.000.

y_2 (0.8202 con 500 pares, 0.8260 exhaustivo) y y_3 (0.8414 y 0.8316) quedan en medio y difieren en 0.0056 en el cálculo exhaustivo, menos de lo que oscilan al cambiar la semilla, así que no los ordeno entre sí. El score de y_2 carga además la decisión sobre las etiquetas -1: leídas como un grupo más, que es la lectura literal del enunciado, sus 497 datos sin asignar entran al promedio intra con una distancia media de 2.8996, y descartando esos pares el mismo y_2 baja a 0.6448. No es que el ruido empuje el score hacia 1 por ser ruido: es que estos 497 puntos resultaron dispersos entre sí, y tratarlos como grupo mete esa dispersión en el promedio intra.

El control barajado es lo que más me sirvió del ejercicio. Da 1.000 en las cuatro particiones, con tamaños de grupo tan distintos como los de y_1 y los de y_0 , así que lo que el score mide por debajo de 1 es estructura y no reparto de tamaños. Lo que hace un grupo gigante es acotar cuánta señal puede haber, porque los pares intra que aporta se parecen a los pares al azar. Reportar el score junto con el reparto de tamaños y el valor de control me deja saber cuánto de él es estructura.